Presentation script: CacheWise replication and KV cache recovery

This script follows the ten-slide deck and its 10:00 timing plan. Questions follow the talk. Read the spoken paragraphs; italic instructions are delivery cues and should not be read aloud. Pause briefly after the main numbers and use the slide transitions to check your timing.

Slide 1 — CacheWise replication and KV cache recovery

0:00–0:30 · 30 seconds

Face the audience. Introduce the three parts without reading each card.

Hello everyone. Our project looks at how to serve coding agents efficiently when GPU memory is limited.

I'll cover three parts: an adapted replication of CacheWise, the cost of recovering evicted cache state, and a host-memory extension with fetch-aware scheduling.

The central question is: when an agent returns from a tool call, how can we reuse its previous work efficiently?

Slide 2 — The same prefix is useful again

0:30–1:25 · 55 seconds

Follow the arrows from the model request, through the tool, to the next request.

A coding agent repeatedly asks the model to act, runs a tool, and sends another request. That next request often contains much of the same context.

The KV cache stores attention keys and values, allowing the model to reuse that context. Without a cached copy, it must process the missing prefix again. That processing is called prefill.

While the tool runs, the cached state still occupies GPU memory. Evicting it creates room for other sessions, but adds recovery work later.

We can recover by recomputing the prefix or restoring a saved copy. The scheduler decides which request goes first, so scheduling, eviction, and recovery affect one another.

Slide 3 — Replication separates two decisions

1:25–2:30 · 65 seconds

Point to the two rows and then the two columns of the policy matrix.

For the replication, we separate scheduling from eviction.

We compare first-come, first-served scheduling with prefix-aware scheduling, which favors requests needing less additional KV state. We also compare least recently used eviction, or LRU, with predictive eviction based on tool names.

All four combinations use the same engine fork, fresh servers, and identical selected inputs.

The setup uses a fourteen-billion-parameter Qwen coding model on one Blackwell GPU. The cache holds about 288,000 tokens.

We test five workload settings: three nested loads in cohort A, plus two additional held-out cohorts. With three repeats per policy, that gives sixty runs and 6,384 request executions.

Our main metric is mean session serving time: the sum of model-request latencies within a session, excluding tool waits.

Slide 4 — Prefix / LRU has the lowest mean

2:30–3:50 · 80 seconds

Trace the green line. Pause after “all five settings.”

This graph shows cohort A as the selected workload grows from ten to thirty concurrent segments. Lower is better.

The blue and orange lines use first-come, first-served scheduling. The green and purple lines use prefix-aware scheduling.

The largest improvement comes from changing the scheduler. Prefix-aware LRU lowers the mean by about 58 percent at ten segments, 62 percent at twenty, and 65 percent at thirty, compared with first-come, first-served LRU.

The report also includes the additional B and C cohorts. Prefix-aware LRU has the lowest mean in all five settings.

The error bars show the observed range of three repeats. They are not confidence intervals. Also, increasing the load adds new segments, so workload composition changes too.

The result supports prefix-aware scheduling in this replay. Next, we can ask whether predictive eviction adds to that benefit.

Slide 5 — Prediction depends on the scheduler

3:50–4:55 · 65 seconds

Point below zero for orange, then above zero for purple. Finish at the P99 callout.

Here, each point compares predictive eviction with LRU within the same repeat block. Below zero means predictive eviction is faster.

Every orange point is below zero: prediction helps first-come, first-served scheduling. Every purple point is above zero: prediction makes prefix-aware scheduling slower.

Across the five settings, the prefix-aware mean increases by about ten to forty-three percent when prediction is added. Its benefit therefore depends on the scheduler.

There is also a tail-latency tradeoff. At the largest cohort A load, prefix-aware LRU improves the session mean, but the average per-run request P99 rises from about 713 to 1,292 seconds.

So a lower mean does not guarantee better responsiveness for the slowest requests. Both measures matter when choosing a policy.

Slide 6 — Preserve evicted KV in host DRAM

4:55–5:55 · 60 seconds

Follow the save arrow to host memory, then the restore arrow back to the GPU.

I’ll now move to the separate Mini-SGLang host-recovery study.

Instead of discarding every evicted prefix, the implementation can save its KV pages in pinned host memory. A shared prefix index tracks whether pages are on the GPU or in the host cache.

When a request returns, the engine restores a compatible host copy before prefill. If no copy survives, it recomputes.

Clean host copies can be retained, avoiding another save when those GPU pages are evicted again. Predictive prefetch can also start restoration before a session returns.

This study keeps twenty-four gibibytes of GPU KV capacity and adds thirty-six gibibytes of host KV capacity. The implemented backing tier is host DRAM; SSD storage remains future work.

Slide 7 — Host restoration reduces session time

5:55–7:10 · 75 seconds

Read the cost condition, then point to each pair of bars. Pause after each ratio.

The cost condition is straightforward: saving, restoring, and managing a copy should cost less than recomputing the missing prefix.

Here, scheduling and eviction stay fixed at first-come, first-served and LRU. We change the recovery method.

At twenty-eight concurrent episodes, reported mean session time falls from 821 to 90.7 seconds, a ratio of about 9.1. At thirty-six episodes, it falls from 1,060 to 222 seconds, a ratio of about 4.8.

The reported missing-prefix deficit also decreases, consistent with more reusable state surviving.

These comparisons include the extra host capacity, and each bar represents one reported run. They show favorable host-memory operating points rather than an equal-total-memory policy comparison.

This study also uses a different engine and replay setup, so we keep its timings separate from the replication results.

Slide 8 — Fetch-aware ordering discounts host hits

7:10–8:30 · 80 seconds

Point to the priority cost, then compare the prefetch and fetch-aware bars.

A request with a host copy is different from a complete cache miss. Fetch-aware scheduling makes that distinction explicit.

Its priority combines tokens needing new prefill with a discounted cost for tokens restored from the host. The discount factor is 0.05, so lower estimated recovery cost goes first. This is a heuristic rather than a measured latency predictor.

At thirty-six episodes, predictive eviction with prefix scheduling records 158 seconds. Adding prefetch alone records 172 seconds. The full fetch-aware combination records 136 seconds.

The diamonds show additional observations: 175 seconds for the predictive-prefix control and 135 for the full combination. The two descriptive means are 166.5 and 135.5 seconds.

These observations favor the full combination, but there are too few repetitions to establish statistical significance or isolate every component.

The design lesson is to consider cache location when ordering requests. Prefetching alone does not guarantee an improvement.

Slide 9 — Keep the conclusions within the evidence

8:30–9:20 · 50 seconds

Move across the three limitation cards without reading every line.

Three limits frame these findings.

First, the replication uses synthetic content and selected workloads. One preemption violated the original audit rule. We retained it under a documented post hoc amendment; its extra computation is unmeasured.

Second, the host study uses a separate replay and adds memory capacity. Most configurations have one reported observation, so its policy comparisons are descriptive.

Third, we have no SSD implementation or measurements. An SSD extension would need storage reads and writes, bounded staging, compatibility metadata, and failure handling.

The completed evidence therefore concerns this adapted replication and host-DRAM recovery.

Slide 10 — Three takeaways

9:20–10:00 · 40 seconds

Deliver each takeaway directly to the audience. Leave a short pause before inviting questions.

To conclude, there are three takeaways.

First, scheduling and eviction interact. Prefix-aware LRU has the lowest mean in all five replication settings, while predictive eviction changes direction with the scheduler.

Second, saving KV state in host memory can avoid costly recovery, with large reported gains when backing capacity is added.

Third, request ordering should account for where reusable state resides. Fetch-aware scheduling is promising here, while SSD evaluation remains future work.

Thank you. I’m happy to take questions.